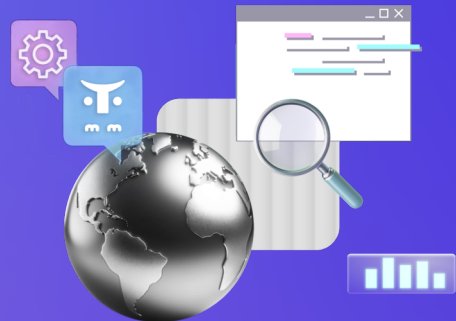


Расчёт ресурсов (Sizing) для приложений и данных

AI Архитектор



● REC

Напишите «+» в чат, если меня слышно и видно





AI Архитектор

Лебедев Михаил

Руководитель направления, PhD

- с 2002 по 2012 системный администратор;
- с 2012 по 2018 системный архитектор, руководитель проектов, технический писатель, разработчик;
- с 2018 по 2026 Data Scientist;
- с 2022 Преподаватель (Доцент ВШЭ, OTUS)

 @MichaelSwan

Правила вебинара



Активно участвуем



Задаем вопросы в чат



Вопросы вижу в чате,
могу ответить не сразу



Запись вебинара придёт на почту

Маршрут вебинара

Оценка потребности в ресурсах

Влияние RPS, SLA, бюджета на выбор архитектуры

Расчет TCO

Практика 1, 2

Рефлексия



Методология расчета ресурсов для stateless/stateful-сервисов, баз данных

Stateless и Stateful-сервисы

Тип сервиса	Описание	Примеры	Ресурсы
Stateless	Не сохраняет состояние между запросами	REST API, простые веб-приложения	Легко масштабируются горизонтально Расход CPU и RAM зависит от количества RPS
Stateful	Хранят состояние между запросами (например, сессии, кэши, данные в памяти)	Кэширующие слои (Redis), приложения с сессиями, базы данных	Требует больше памяти и синхронизации, сложнее масштабируются



Свойства баз данных

База данных	SQL	NoSQL	Vector-базы
Подходящие типы данных	Структурированные данные (табличные, со схемой), реляционные данные (связи между таблицами).	Полуструктурированные и неструктурированные данные (JSON, XML, документы, графы).	Векторы (embeddings) — часто используются в машинном обучении, рекомендательных системах, NLP.
Требование	Сложные JOIN-запросы	Быстрое чтение/запись большого количества записей	Поиск по векторам, близость, косинусное расстояние
Масштабируемость	Вертикально и горизонтально, но сложнее	Легко горизонтально (sharding, replication)	Горизонтальная, с поддержкой индексов
Производительность	Хорошая для сложных запросов	Высокая для однотипных операций	Зависит от размера векторов и индексов
Подходящие операции	Смешанные операции (CRUD), сложные запросы	Массовая запись, быстрое чтение	Поиск по векторам, близость, добавление новых векторов



Свойства баз данных

База данных	SQL	NoSQL	Vector-базы
Поддержка инструментов	Интеграция с BI-инструментами, ORM, ETL-процессами	Хорошо работает с NoSQL-фреймворками, асинхронными системами	Интеграция с ML-фреймворками (TensorFlow, PyTorch, Hugging Face)
Управляемость	Требует настройки, поддержки, бэкапов	Часто управляемые (Yandex Cloud, AWS, GCP)	Требует специфических знаний
Стоимость	Средняя	Низкая	Средняя-высокая
Примеры использования	Банки, CRM, ERP, системы с высокой согласованностью	Мессенджеры, соцсети, IoT, веб-приложения	Рекомендательные системы, NLP, компьютерное зрение, ML
Название	PostgreSQL, MySQL, MSSQL	MongoDB, Cassandra, DynamoDB	Weaviate, Pinecone, FAISS, Milvus
Характеристики	- Зависит от: - Объема данных (диск), - Частоты чтения/записи, - Индексации, - Типа запросов (OLTP vs OLAP), - Нужды в репликации и бэкапах.	- Хорошо масштабируются горизонтально. - Подходят для неструктурированных данных. - Расход памяти и CPU зависит от количества записей и скорости чтения/записи.	- Используются для работы с векторами (например, embeddings). - Требуют большого объема RAM и дискового пространства. - Влияние RPS — критично.



Расчет ресурсов для SQL

Условия:

- **Объём данных:** 1 ТБ
- **Частота запросов:** 1000 запросов/с
- **Тип нагрузки:** смешанная (читать/писать)
- **Требования к производительности:** <100 мс ответ
- **Резервное копирование:** ежедневное
- **Репликация:** 2 реплики

Ресурс	Основная нода	Реплика	Итого (3 ноды)
CPU	16 vCPU	8 vCPU	32 vCPU
RAM	64 ГБ	32 ГБ	128 ГБ
Диск (SSD)	2 ТБ	1 ТБ	4 ТБ
Сеть	1 Гбит/с	1 Гбит/с	3 Гбит/с



Расчет ресурсов для NoSQL

Условия:

- **Объём данных:** 5 ТБ
- **Частота запросов:** 5000 запросов/с (читать/писать)
- **Тип нагрузки:** высокая производительность, масштабируемость
- **Репликация:** 3 ноды
- **Индексация:** частичная
- **Требования к согласованности:** eventual consistency

Ресурс	Основная нода	Реплика	Итого (4 ноды)
CPU	8 vCPU	8 vCPU	32 vCPU
RAM	128 ГБ	128 ГБ	512 ГБ
Диск (SSD)	10 ТБ	5 ТБ	25 ТБ
Сеть	10 Гбит/с	10 Гбит/с	40 Гбит/с



Расчет ресурсов для Vector DB

Условия:

- **Объём векторов:** 100 млн
- **Размерность вектора:** 768
- **Частота запросов:** 1000 векторных запросов/с (ANN)
- **Тип нагрузки:** высокая производительность, поиск по векторам
- **Требования:** высокая точность, поддержка индексов

Ресурс	Основная нода	Реплика	Итого (3 ноды)
CPU	16 vCPU	8 vCPU	32 vCPU
RAM	256 ГБ	128 ГБ	512 ГБ
Диск (SSD)	10 ТБ	10 ТБ	30 ТБ
Сеть	10 Гбит/с	10 Гбит/с	30 Гбит/с



**Влияние RPS, заданного уровня
качества, финансовых
ограничений и других
требований**

Основные параметры для расчета

Параметр	Описание
RPS (Requests Per Second)	(Запросы в секунду) — ключевая метрика производительности веб-серверов и приложений, показывающая количество обрабатываемых запросов за 1 секунду.
Тип нагрузки	Смешанная (читать/писать), только чтение, только запись.
Среднее время обработки запроса	Влияет на необходимое количество потоков/процессов.
Требования к SLA	Уровень доступности, latency, отказоустойчивость.
Финансовые ограничения	Бюджет на оборудование, обслуживание, облако.



Влияние RPS на ресурсы

Параметр	Влияние
CPU	Чем выше RPS, тем больше потоков и CPU
RAM	Зависит от кэширования и буферов
Disk	Важно для БД и логирования. При высоком RPS может потребоваться SSD
Сеть	Нужна при распределенном хранении и обработке



Требования к уровню качества (QoS / SLA)

Параметр	Влияние
SLA (Service Level Agreement)	Например, 99.9% uptime, latency < 200ms.
Уровень отказоустойчивости	Количество реплик, бэкапов, HA-конфигурации
Гибкость	Возможность масштабирования, автоматическое резервное копирование



Финансовые ограничения

Параметр	Влияние
Бюджет на оборудование	Серверы, хранилища, сеть
Стоимость обслуживания	Аренда, ремонт, обновления, лицензии
Стоимость развертывания	Настройка, тестирование, мониторинг

Расчет TCO для on-premise

Расчет TCO

TCO (Total Cost of Ownership) — это общие затраты на владение и эксплуатацию системы в течение всего жизненного цикла.

Формула TCO:

$$\text{TCO} = \text{Стоимость оборудования} + \text{Стоимость обслуживания} +$$
$$\text{Стоимость энергопотребления} + \text{Стоимость человеческого ресурса} +$$
$$\text{Стоимость рисков (выход из строя, downtime)}$$



Расчет ресурсов для компонентов учебного проекта при развертывании on-premise и в облаке.

Учебный проект: Онлайн-платформа для тестирования студентов

Состав проекта:

1. Фронтенд (Stateless) – веб-приложение (React, Vue).
2. Бэкенд (Stateless) – REST API (Python/Flask, Node.js).
3. База данных (SQL) – PostgreSQL.
4. Кэширующий слой (Stateful) – Redis.
5. Векторная БД (Stateful) – Weaviate (для обработки векторов).
6. Логирование и мониторинг – ELK stack.

Оценка RPS:

- Ожидаемое количество пользователей: 5000
- Среднее RPS: 100
- Пиковая нагрузка: 300 RPS



Расчет ресурсов для каждого компонента

1. Фронтенд (React)

- Stateless, легко масштабируем.
- Ресурсы:
 - CPU: 0.5–1 core
 - RAM: 512–1024 МБ
 - Disk: 10 GB

2. Бэкенд (Python/Flask)

- Stateless, обрабатывает бизнес-логику.
- RPS: 100–300
- Ресурсы:
 - CPU: 2–4 core
 - RAM: 2–4 GB
 - Disk: 20 GB

3. База данных (PostgreSQL)

- Объем данных: 100 GB
- Частота запросов: 100 RPS
- Индексация: средняя
- Ресурсы:
 - CPU: 4–8 core
 - RAM: 8–16 GB
 - Disk: 200–300 GB (SSD)

4. Кэширующий слой (Redis)

- Хранение сессий и кэша.
- Ресурсы:
 - CPU: 1–2 core
 - RAM: 2–8 GB (в зависимости от объема кэша)
 - Disk: 25–50 GB (для бэкапов)

5. Векторная БД (Weaviate)

- Обработка векторов (например, для рекомендаций).
- Объем данных: 100М векторов
- Ресурсы:
 - CPU: 8–16 core
 - RAM: 16–32 GB
 - Disk: 500–1000 GB (SSD)

6. Логирование и мониторинг (ELK stack)

- Логирование, графики, алерты.
- Ресурсы:
 - CPU: 2–4 core
 - RAM: 4–8 GB
 - Disk: 500–1000 GB (для логов)



Развертывание on-premise

Компонент	vCPU	RAM Gb	Disk Gb	Стоимость тыс руб
Фронтенд (React)	1	1	10	50
Бэкенд (Node.js)	4	4	20	100
БД (PostgreSQL)	8	16	300	200
Кэш (Redis)	2	2	50	80
Логирование (ELK)	4	8	1 000	150
Weaviate (vector Db)	16	32	1 000	400
Итого				980



Физические Развертывание в облаке (Yandex Cloud, AWS, Google Cloud)

Компонент	Тип	Стоимость (тыс руб/мес)
Фронтенд	t2.nano	1
Бэкенд	t2.medium	3
PostgreSQL	db.r4.large	15
Redis	4 GB RAM	5
Weaviate	16 vCPU / 32 GB RAM	40
ELK	m4.large	8
Итого		72

Выводы

Параметр	On-premise	Cloud
Стоимость	Высокая, но разово 980 000 руб	Низкая, но ежемесячно 72 000 руб/мес
Управление	Требуется IT-штат	Автоматизировано
Гибкость	Низкая	Высокая
Масштабируемость	Требует времени	Автоматическая
SLA	Зависит от настройки	Обеспечивается провайдером
Итого	Подходит, если бюджет позволяет и проект стабилен	Более гибкое и управляемое решение, особенно при росте нагрузки



Расчет ресурсов для Vector DB

Условия:

- Набор данных — [LAION-400M](#)
- Объём векторов: 400 млн
- Размерность вектора: 512
- Тип данных FLOAT16 (вместо FLOAT32 без значительных потерь точности)
- Частота запросов: 5000 векторных запросов/с

Ресурс	Основная нода	Описание
Системная память	8.34 ГБ	Память, зарезервированная для внутренних систем и ОС, не зависит от размера набора данных.
Память данных	39.27 ГБ	Резидентная память процесса Vector DB, ее нельзя выгрузить
Кеш-память	14.54 ГБ	Дисковый кеш, который использует Vector DB. Необходим для быстрого поиска
IdTracker	12.4 ГБ	Хранение идентификатора и версии
Итого	54.12 ГБ	Общее использование оперативной памяти сервером Vector DB

Расчет параметров ОЗУ

Пример расчета ОЗУ модели.

- Модель на HuggingFace (Qwen, Llama, deepseek).
- Квантизация модели (32 bit, 16 bit, 8 bit, 4 bit).
- GPU.

Параметры модели

Веса занимают основной объем памяти и рассчитываются как число параметров N * размерность M .

Например: FP16/BF16 → 2 байта на параметр, INT8 → 1 байт, INT4 → 0,5 байта.

Пример: Qwen-7B ($N = 7\,000\,000\,000$) в FP16 ($M = 2$ байта) ≈ 14 ГБ VRAM только под веса.

Активации — это промежуточные результаты работы слоев. Их объем зависит от batch size, длины последовательности (context length) и архитектуры. Обычно это **несколько** дополнительных **гигабайт**.

Расчет параметров ОЗУ

Tensor Type	TF32/FP32	BF16/FP16	INT8	INT4
Размер параметра	4 байта	2 байта	1 байт	~0,5 байта
VRAM весов QwQ-32B	128 ГБ	64 ГБ	32 ГБ	16 ГБ
KV-cache QwQ-32B (1 batch)	10 ГБ	5 ГБ	2,5 ГБ	1,2 ГБ
Total VRAM GB	~138 ГБ	~70 ГБ	~35 ГБ	~18 ГБ

Пример анализа нагрузки

Анализ нагрузки

Оценить текущую нагрузку на систему, выявить узкие места, определить необходимость масштабирования и рекомендовать действия для повышения устойчивости и производительности.

Параметр	Значение
Ожидаемая RPS	[X]
Фактическая RPS	[Y]
CPU (средняя загрузка)	[Z %]
RAM (использование)	[A GB]
Disk I/O	[B IOPS]
Latency (среднее время ответа)	[C ms]
Error rate (ошибки 5xx)	[D %]
SLA требования	[например, 99.9% uptime, latency < 200ms]
Тип сервиса	[stateless/stateful]



Выявленные проблемы

- Загрузка CPU: [описание, например, "CPU загружён на 85%, что близко к пределу"]
- Память: [описание, например, "RAM используется на 90%, возможен OOM-kill"]
- Диск: [описание, например, "IOPS на диске достигают 95% от максимальных"]
- Сеть: [описание, например, "сетевой трафик в пике превышает 80% от лимита"]
- Latency: [описание, например, "среднее время ответа увеличилось на 30%"]
- SLA: [описание, например, "SLA не нарушено, но близко к границе"]

Проблема	Причина	Влияние
Высокая загрузка CPU	Пиковая нагрузка	Падение производительности
Недостаток RAM	Недостаток ресурсов	Возможные ошибки OOM
Недостаточная пропускная способность диска	Интенсивные операции чтения/записи	Падение скорости обработки
Слишком высокий latency	Перегрузка бэкенда	Плохой пользовательский опыт



Рекомендации

Рекомендация	Описание	Приоритет
Увеличить количество бэкенд-серверов	Горизонтальное масштабирование	Важно
Увеличить объем RAM	Вертикальное масштабирование	Средне
Внедрить кэширование	Использовать Redis для часто запрашиваемых данных	Важно
Оптимизировать запросы к БД	Добавить индексы, уменьшить количество запросов	Важно
Внедрить автомасштабирование	Настроить автоматическое увеличение ресурсов	Важно



Прогноз

Период	Ожидаемая RPS	Текущие ресурсы	Требуемые ресурсы	
Месяц 1	500	Достаточно	—	
Месяц 2	700	Недостаточно	Добавить 1 инстанс	
Месяц 3	1000	Недостаточно	Добавить 2 инстанса и увеличить RAM	

- Текущая нагрузка приближается к пределам возможностей.
- Рекомендуется начать горизонтальное масштабирование.
- Необходимо внедрить мониторинг и алертинг для оперативного реагирования.



Ваши вопросы



если есть вопросы



если вопросов нет

